Efficient Secure Inference Scheme in Multiparty Settings for Industrial Internet of Things

Jie Lin, Yinbin Miao, *Member, IEEE*, Linfeng Wei, Tao Leng, and Kim-Kwang Raymond Choo, *Senior Member, IEEE*

***Abstract*—Secure inference is the main technology to avoid privacy leakage when reasoning with machine learning models in multiparty settings for industrial Internet of Things (IoT). Existing solutions based on secure multiparty computation have been extensively explored in both academic and industrial fields, but these solutions are deployed in single user single model provider setting. When multiparty collaborate to handle tasks in multiuser multimodel provider setting, these solutions are no longer applicable as the mechanism of the linear computation protocol restricting the number of parties increases. In addition, these existing schemes need to conduct multiple checks during the inference of malicious models, which results in additional communication overhead. To solve these issues, we propose a multiparty secure inference scheme by using an improved multiplication protocol for achieving matrix multiplication, which achieves efficient linear calculation and supports adaptive parties. We also design a new check protocol to inspect calculation results of all layers with just one execution, which achieves lower communication and calculation overheads. Formal security analysis proves that our scheme achieves malicious security in the honest-majority setting, and extensive experiments demonstrate that our scheme reduces the communication costs by about 30% in large neural networks when compare with previous solutions.**

***Index Terms*—Privacy preserving, secret sharing (SS), secure inference, secure multiparty computation.**

Manuscript received 23 July 2023; revised 4 May 2024; accepted 30 May 2024. Date of publication 1 July 2024; date of current version 7 October 2024. The work of Kim-Kwang Raymond Choo was supported by the Cloud Technology Endowed Professorship. This work was supported in part by the National Natural Science Foundation of China under Grant 62072361, in part by the Shaanxi Fundamental Science Research Project for Mathematics and Physics under Grant 22JSY019, in part by the National Natural Science Foundation of China under Grant 62125205 and Grant U23A20303, in part by the Fundamental Research Funds for the Central Universities under Grant QTZX24077, in part by the Opening Project of Intelligent Policing Key Laboratory of Sichuan Province under Grant ZNJW2023KFMS002, and in part by the Open fund of Key Laboratory of Computing Power Network and Information Security under Grant 2023ZD020. Paper no. TII-23-2763. *(Corresponding authors: Yinbin Miao; Tao Leng.)*

Jie Lin and Yinbin Miao are with the School of Cyber Engineering, Xidian University, Xi'an 710071, China (e-mail: jielin@stu.xidian.edu.cn; ybmiao@xidian.edu.cn).

Linfeng Wei is with the School of Cyber Security, Jinan University, Guangzhou 510632, China (e-mail: twei@jnu.edu.cn).

Tao Leng is with the Intelligent Policing Key Laboratory of Sichuan Province, Sichuan Police College, Luzhou 646000, China, also with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100085, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: lengtao@iie.ac.cn).

Kim-Kwang Raymond Choo is with the Department of Information Systems and Cyber Security, University of Texas at San Antonio, San Antonio, TX 78258 USA (e-mail: raymond.choo@fulbrightmail.org).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TII.2024.3413324.

Digital Object Identifier 10.1109/TII.2024.3413324

I. Introduction

IN THE industrial Internet of Things (IoT) [1], the applications of Artificial Intelligence (AI) are extensive, such as smart manufacturing, quality control, and medical IoT, but it is also accompanied by the problem of privacy leakage. For example, the use of unprotected data in fault diagnosis systems in the industrial IoT leads to the leakage of fault diagnosis models and industrial confidential information. Secure inference based on Secure Multiparty Computation (SMC) [2] can solve this problem, which allows multiple parties to jointly perform AI reasoning calculations without leaking any party's data. A variety of secure inference schemes [3], [4], [5], [6] based on SMC have been proposed, but these schemes still have the following two problems.

As shown in Fig. 1, the first problem is that existing solutions are deployed in single user single model provider setting, but when there are multiuser and multimodel providers [Fig. 1(d)], such as industrial automation [7] and intelligent logistics, existing solutions are no longer applicable as they cannot tolerate so many parties. Even though some of the three-party or four-party solutions can also be used in multiuser multimodel provider setting by adding one or two servers, respectively [Fig. 1(b) and (c)]. For example, [8] was deployed in a three-party setting, which still can provide inference service even if a server is attacked. The authors in [5] and [9] customized a four-party linear computing protocol with lower communication rounds, which reduce communication overhead. However, as the mechanism of the linear computation protocol restricting the number of parties increases, the above schemes cannot be deployed in multiuser multimodel provider setting.

The second problem is that the check protocols of existing schemes incur a lot of additional communication and calculation overheads. Malicious adversaries may generate wrong intermediate values during the calculation process. To ensure the correctness of the final results, a secure inference scheme must contain the check protocol. The existing schemes usually use MAC codes [10], zero-knowledge proofs or related random values [5] to verify computation results. But the number of check operations increases linearly with the number of layers,

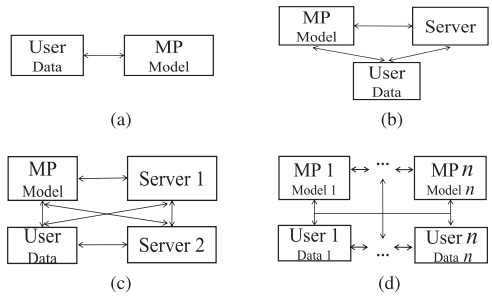


Fig. 1. Some possible settings (MP: Model Provider). (a) 2 Parties. (b) 3 Parties. (c) 4 Parties. (d) n Parties.

which brings more computation and communication overheads in the deep model with many layers. In practice, there will not be frequent errors, so this detection mechanism is somewhat wasteful and incurs unnecessary overhead.

To solve the above two issues, we propose a multiparty secure inference scheme supporting adaptive number of parties. Specifically, we use an improved DN (I.Damgård and J.B.Nielsen) multiplication protocol [11] based on replication secret sharing to achieve matrix multiplication, which achieves efficient linear calculation and supports adaptive parties without switching inference protocols according to the number of parties. We also design a new check protocol to guarantee the correctness of calculation results of all layers with just one execution, which reduces the communication and calculation overhead. Our concrete contributions are as follows.

1) We propose an efficient multiparty secure inference scheme based on replicated secret sharing by using an improved DN multiplication protocol to achieve matrix multiplication, which achieves efficient linear calculation and supports adaptive parties.
2) We design a new check protocol that only needs to be executed once to inspect the calculation results of all layers in the entire network, which achieves lower communication and calculation costs.
3) We formally prove that our scheme achieves fairness in malicious model. We also perform extensive experiments to demonstrate that our scheme saves about 30% communication overhead compared with the state-of-the-art secure inference framework.

The rest of this article is organized as follows. Section II introduces the related work. Section III describes the cryptographic primitives used in our scheme. Section IV presents the problem formulation of our protocol. In Section V, we provide a detailed description of the two protocols in our scheme. Section VI covers the implementation and evaluation of our protocol. Finally, Section VII concludes this article.

II. Related Works

In this section, we introduce related work from two aspects of computational efficiency and security.

Computational efficiency: Juvekar et al. [12] proposed a framework by using Homomorphic Encryption (HE) and Garbled Circuits (GC) to compute the linear layer and nonlinear layer respectively, which solves the inefficiency of using a general SMC framework for multiparty inference [8], [13], [14]. But the encryption and decryption operations of HE are concentrated in the online stage, which makes users wait a little longer. Mishra et al. [15] moved the complex HE operations to the offline phase and partially replaced the ReLU function with a computationally friendly polynomial approximate activation function in the nonlinear layer, which greatly reduces the prediction latency and improves user experience. Next, many subsequent three-party and four-party schemes can prove that secret sharing (SS) has better performance. Mohassel et al. [3] used the general SMC framework based SS to invent a framework in the three-party setting, which proves that SS has better computing performance but require more security settings. To reduce the cost of security settings, Patra et al. [6] proposed a scheme by putting the computation-intensive distributed zero-knowledge of online phase into preprocessing phase, which supports the fast online reasoning. Koti et al. [4] constructed a framework SWIFT by using many new protocols based on SS, which achieves robustness while being as fast as [6] in three-party setting. Based on [4], Chaudhari et al. [9] designed a more efficient framework in four-party setting without using expensive public key primitives and broadcast communication. Koti et al. [5] designed a framework by redesigning a communication-efficient multiplication protocol with multiple inputs and using a GC-based end-to-end transformation technique, which solves the problem of constant conversion between the arithmetic circuit and the GC.

Security: A semihonest adversary will perform as required by the protocol, while a malicious adversary can deviate from the protocol arbitrarily. Depending on the protocol's ability to resist the adversary, the security of the protocol can be categorized into semihonest and malicious security, where malicious security can also be categorized into abort security, fairness, and robustness from weak to strong. Fairness means that if the corrupt party can get the output then all the parties can get the output, but the malicious adversary may refuse to send the last message to the honest party, so in order to cover this attack a slightly weaker abort security is proposed. Robustness ensures that the output can be delivered to all honest parties. Lehmkuhl et al. [10] proposed an abort scheme by designing a conditional disclosure secret protocol and using MAC codes and zero-knowledge proofs to verify the linear layer, which provides a lot of inspirations on how to design the verification module. But their scheme only assumes that client has malicious behavior. Chandran et al. [16] used cheap oblivious transfer and one-time PAD encryption, which achieves malicious security at the cost of a semihonest setup in [10]. When the number of parties is set as 4, the detection method changes to using specially designed related randomness since the HE is replaced by SS. Koti et al. [5] used the generated randomness in the calculation of linear layers and sent to other parties to reconstruct the related values to check the correctness. The nonlinear layer of [5] used a customized GC and a robust joint send protocol to send labels to the evaluator, which ensures that someone sending the wrong message will be caught.

The comparison between the above secure inference frameworks is shown in Table I. The dot product was chosen as a

TABLE I
COMMUNICATION COMPARISON OF DOT PRODUCTS FOR SMC FRAMEWORKS

Reference	Parties	Security	Method	Communication of dot product		
				Offline	Online	Total
[15]	2	Semihonest	HE+SS+GC	$2e$	1ℓ	$3e+1\ell$
[10]	2	Abort	HE+SS+GC	$4e$	1ℓ	$4e+1\ell$
[16]	2	Abort	HE+SS+GC	0	$5e$	$5e$
[4]	2	Robust	SS	3ℓ	3ℓ	6ℓ
[6]	3	Fair	SS	3ℓ	3ℓ	6ℓ
[3]	3	Abort	SS+GC	$12k\ell$	$9k\ell$	$21k\ell$
[4]	4	Robust	SS	3ℓ	3ℓ	6ℓ
[9]	4	Fair	SS+GC	3ℓ	3ℓ	6ℓ
[5]	4	Fair/Robust	SS+GC	2ℓ	3ℓ	5ℓ
Our	4	Fair	SS	0	4ℓ	4ℓ
[3]	n	Abort	SS+GC	$4kn\cdot\ell$	$3kn\cdot\ell$	$7kn\cdot\ell$
Our	n	Fair	SS	0	$(n+d-1)\ell$	$(n+d-1)\ell$

Notes: Parties: Number of computing parties; Security: Safety performance, from low to high is Semi-honest, Abort, Fair, Robust; e: Ciphertext length; ℓ: Size of ring in bits; k: Length of the vectors; d: Number of corrupted parties.

parameter because it is the most important building block in secure inference applications.

III. PRELIMINARIES

A. Replicated Secret Sharing

Replicated Secret Sharing [11]: Given a set of parties $\mathcal{P} = \{P_1, \ldots, P_n\}$, there are at most d corrupted parties, such that $d < n/3$, so the number of honest parties is $(n-d)$, and there are a total of $\binom{n}{n-d}$ possibilities for the set without corrupted parties. So a value can be divided into $\binom{n}{n-d}$ shares by letting each set that may contain only honest parties hold a share. We denote a replicated secret sharing (RSS) of a value x with threshold d by $[\![x]\!]_d$. We let $k = \binom{n}{n-d}$ and $T_1, \ldots, T_k \subset \mathcal{P}$ be all subsets of parties with size $n-d$. The RSS scheme is defined by the following two algorithms.

1) *Share*(x, d)*:* The dealer inputs a secret x with threshold d and generates k random shares $x_1, \ldots, x_k \in R$ such that $x_1 + \cdots + x_k = x$. For each $j \in [1, k]$, the dealer sends share x_j to subset T_j, so the party $P_i \in T_j$ receives x_j for $i \in [1, n]$. Since P_i exists in $\lambda = \binom{n-1}{d}$ subsets, P_i can receive λ shares. This λ shares received by P_i form a vector $\vec{x_i}$. Thus, we can get $[\![x]\!]_d = \{\vec{x_1}, \ldots, \vec{x_n}\}$.
2) *Reconstruct*$([\![x]\!]_d, q)$*:* Parties choose P_q to reconstruct the secret sharing according to the input q. For each $j \in [1, k]$, if the set $T_j \not\ni P_q$, then every party in T_j sends his all shares to P_q. For each subset T_j holding a share x_j, if someone sends wrong x'_j to P_q, P_q will choose x_j with more repetitions. Eventually P_q computes $x = \sum_1^k x_j$.

Let $[\![x]\!]_d$ and $[\![y]\!]_d$ be two consistent shares and let $\alpha \in R$ be some public constants. RSS has the following operations.

$[\![x]\!]_d + [\![y]\!]_d$*:* Let vector $\vec{x}_i$ and $\vec{y}_i$ be the two shares held by P_i, then P_i performs point-wise addition between the two vectors and stores the result as its output.

$\alpha \cdot [\![x]\!]_d$*:* Let vector $\vec{x}_i$ be the share held by P_i, then P_i multiplies each component in $\vec{x}_i$ with α and stores the result as its output.

$\alpha + [\![x]\!]_d$*:* One predetermined subset of parties T_j with $|T_j| = n-d$, which holds x_j and stores the $x_j = x_j + \alpha$. The other $k-1$ shares remain the same.

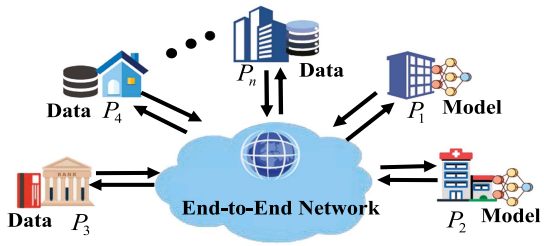


Fig. 2. System model of our scheme.

$[\![x]\!]_d \cdot [\![y]\!]_d$*:* According to [11], we can compute $[\![x]\!]_d \cdot [\![y]\!]_d = [\![x \cdot y]\!]_{2\,d}$, and the specific steps are omitted here.

B. Random Secret Generation

Let $\mathcal{F} = \{F_k \mid k \in \{0,1\}^\kappa, F_k : \{0,1\}^\kappa \to R\}$ be a family of pseudorandom functions, where κ be the security parameter. As shown in [11], all parties can use $[\![k]\!]_d$ to generate corresponding numbers of random shares $[\![r]\!]_d$. To generate the ℓ-th $[\![r]\!]_d$, all parties have each subset T_j holding k_j, then the parties in T_j compute the $r_\ell^T = F_{k_j}(\ell)$. As shown in **Share** algorithm, r_ℓ^T can be composed of $[\![r_\ell]\!]_d$. Using $[\![r]\!]_d$, we have the following:

$\mathcal{F}_{\text{coin}}$ can generate fresh random coins by letting the parties compute a new random sharing $[\![r]\!]_d$ and then open it by running reconstruct $([\![r]\!]_d, i)$ for each $i \in [n]$.

The value x of n-party Additive Secret Sharing (ASS) is divided into multiple shares, namely $x = \langle x \rangle_1 + \ldots + \langle x \rangle_n$.

$\mathcal{F}_{\text{corRand}}$ can generate any number of $([\![r]\!]_d, \langle r \rangle^U)$ for a pseudorandom $r \in R$ and $U \subset \mathcal{P}$ such that $|U| = 2d+1$. $\langle r \rangle^U$ is an additive secret sharing of r across the parties in subset U. According to [11] parties can prepare this correlated randomness without any interaction but a short share step building on the PRSS method of [17].

IV. PROBLEM FORMULATION

A. System and Threat Model

As shown in Fig. 2, $P_1, \ldots, P_n$ represent the parties involved in inference. Some of parties are model providers such as P_1 and P_2, and the rest of the parties are users who have private data $\boldsymbol{x}$. n parties communicate through paired end-to-end networks to ensure the security and reliability of communication. We consider an honest majority setting, where a static malicious adversary corrupts no more than $d < n/3$ parties. These corrupted parties are defined before the protocol starts and not collude, and can arbitrarily deviate from the protocol to access data or model. We do not guarantee data privacy in model inversion, attribute inference, and membership inference, which is an orthogonal issue and beyond the scope of this study.

Model providers and users generate the RSS of trained Neural Network (NN) model $\mathbf{M}$ and data $\boldsymbol{x}$, respectively. Then, all parties perform SMC protocol and communicate through the end-to-end network to output result as $z' = \text{NN}(\text{RSS}(\mathbf{M}) \cdot \text{RSS}(\boldsymbol{x}))$. The intermediate values and inference results in the computation process of the protocol are in the form of secret sharing, and the protocol will send the results to the selected

A set of parties $\mathcal{P} = \{P_1, \ldots, P_n\}$ compute a function f with fairness:
Input: The honest $P_i \in \mathcal{P}$ inputs the function (*input*, x_i), and the corrupted parties input function according to the instructions of the adversary, which may be an arbitrary value or abort. If this function has received (*input*, $*$) from P_i, it will ignore this message, otherwise record x_i.
Output: If there is a x_i = abort, output $\perp$ to all parties, else output $f(x_1, \ldots, x_n)$ to all parties.

Fig. 3. Computation of the fair ideal function f.

party for reconstruction. The accuracy of the secure inference scheme needs to satisfy (1).

$$\Pr(z' = z | z' = \mathrm{NN}(\mathrm{RSS}(\mathbf{M}) \cdot \mathrm{RSS}(\boldsymbol{x})), \quad z = \mathrm{NN}(\mathbf{M} \cdot \boldsymbol{x})) \approx 1. \tag{1}$$

B. Security Model

Similar to previous works [5], [9], we target privacy against a static and malicious Probabilistic Polynomial Time (PPT) adversary following the real-world and ideal-world simulation paradigm [18]. We also consider fairness in malicious security, which implies that if the corrupted party can get the output then all parties can get the output, otherwise all parties cannot get the output. For this reason we set the ideal function f to be fair as shown in Fig. 3, and when f receives an abort message from some honest parties, all parties cannot get the final output. Let $\mathrm{IDEAL}_{f,\mathcal{S}}$ denote the output of the honest parties and ideal-world adversary $\mathcal{S}$ from the ideal function f with the security parameter κ and input x. Similarly, let $\mathrm{REAL}_{\Pi,\mathcal{A}}$ denote the output of the honest parties and PPT real-world adversary $\mathcal{A}$ from the real protocol Π. If for every adversary $\mathcal{A}$, there exists an ideal-world adversary $\mathcal{S}$ that corrupts the same parties and satisfies (2), we say that the secure inference protocol Π securely realizes fair f.

$$\mathrm{IDEAL}_{f,\mathcal{S}}(1^\kappa, x) \approx \mathrm{REAL}_{\Pi,\mathcal{A}}(1^\kappa, x). \tag{2}$$

C. Design Goals

Our secure inference scheme's design goals include the following points.

1) *Scalability:* Our scheme should support multiparty secure inference and adaptive parties count to work in multiuser and multimodel provider settings.
2) *Efficiency:* Our scheme should enable efficient communication and computation to achieve lower usage costs.
3) *Security:* Our scheme should be against malicious adversaries to protect the privacy of data and ensure the correctness of the output results.

V. Proposed Scheme

A. Technical Overview

To achieve efficiency and security, we use two new protocols in secure inference. The first one is the multiplication protocol based on RSS with two-round communication. We first select a set U whose size is only related to d. The parties in set U can locally compute the masked secret share $\langle x \cdot y - r \rangle^U$ to compute $[\![x \cdot y]\!]_d$, which makes our multiplication protocol only need $(n + d - 1)$ ring elements for each dot product.

Another protocol is the check protocol, where each party needs to record the sent and received message msg for multiple multiplications. The first step of check protocol is to determine whether the messages from all parties are consistent. In this step, the parties sample the random coefficients and broadcast a linear combination of the messages they send and receive in all multiplications. If two parties have conflicting views, then one of them must be cheating. If all views are consistent, then the parties jointly calculate $[\![msg']\!]_{2d}$ and confirm the correctness of the msg by determining that $msg' = msg$ after opening $[\![msg']\!]_{2d}$. Ultimately, the check protocol requires only $\binom{n-1}{2d} \cdot 2d + 2$ ring elements.

B. Construction

Our scheme can be divided into two phases: 1) the offline phase; and 2) the online phase. The online phase can be further divided into three categories: 1) linear layer; 2) nonlinear layer; and 3) check output.

1) Offline Phase: The offline phase can be executed independently of user input. Our scheme first counts the total number of parties n and the number of corrupted parties d in this inference. For each layer of the model, the model provider generates $[\![\mathbf{M}]\!]_d$ and sends it to other parties. Finally, all parties call $\mathcal{F}_{\text{corRand}}$ to generate the random number pairs $([\![r]\!]_d, \langle r \rangle^U)$ which are needed in the online phase. Each multiplication operation or each multiplication of vectors consumes a random number pair. In this article, we compute the linear and convolutional layers by multiplication of matrices, thus the number of $([\![r]\!]_d, \langle r \rangle^U)$ required is the number of matrix multiplications multiplied by the number of random numbers required for a single matrix multiplication.

Communication of $\mathcal{F}_{corRand}$: Let the size of the ring be ℓ bits. $P_i \in U$ needs to share $[\![k_i]\!]_d$ to other $n-1$ parties, each of which receives $\binom{n-1}{d}$ shares, and $|U| = 2d + 1$. So the communication of $\mathcal{F}_{\text{corRand}}$ requires $(2d+1)(n-1)\binom{n-1}{d}\ell$ bits.

2) Linear Layer: The main calculation of the linear layer is multiplication, for ease of understanding, we first introduce how to realize the multiplication of two numbers x and y, and finally extend the multiplication of numbers to the multiplication of matrices. Based on the calculation of RSS given in Section III-A, $x \cdot y$ can be calculated, but the result of $[\![x]\!]_d \cdot [\![y]\!]_d$ is $[\![x \cdot y]\!]_{2d}$, then parties subtracts $[\![r]\!]_{2d}$ and sends it to P_1 to recover $xy - r$, P_1 then sends $xy - r$ to all parties who can compute $[\![r]\!]_d + xy - r$ to get $[\![xy]\!]_d$. However, in this process malicious P_1 is able to learn private information from the redundancy of the two multiplication computations. So we use the improved DN protocol and $\mathcal{F}_{\text{corRand}}$ to achieve linear layers. The multiplication algorithm is shown in Fig. 4. The set U is a predetermined set with $|U| = 2d + 1$. Each party $P_i \in U$ initializes $z_i = 0$. For different j, k, let $T \subset U$ be the set of parties holding both x_j and y_k and let P_ℓ be the party with the smallest index in T. Then, P_ℓ computes $z_\ell \leftarrow z_\ell + |T| \cdot (x_j \cdot y_k)$, whereas other party P_u

$[\![x]\!]_d \odot_U [\![y]\!]_d$: Let U be the set$\{P_1, ..., P_{2d+1}\}$.
Input: The parties input $[\![x]\!]_d$ and $[\![y]\!]_d$
Output: $\langle z\rangle^U$
The protocol:

- Each party $P_i \in U$ initializes $z_i = 0$.
- For each pair of x_j and y_k, let $T \subset U$ be the set of parties holding both x_j and y_k and let P_ℓ be the party with the smallest index in T. Then, P_ℓ sets: $z_\ell \leftarrow z_\ell + |T| \cdot (x_j \cdot y_k)$, whereas each $P_u \in T$ with $u \neq \ell$ sets: $z_u \leftarrow z_u - (x_j \cdot y_k)$.
- Each party $P_i \in U$ stores $\langle z\rangle^U = z_i$ as their output.

Fig. 4. $[\![x]\!]_d \odot_U [\![y]\!]_d$ multiplication algorithm.

Protocol 1: Π_{Mult} Multiplication Protocol.

Input: The parties input $[\![x]\!]_d$ and $[\![y]\!]_d$
Output: $[\![z]\!]_d$

1: The parties call $\mathcal{F}_{\text{corRand}}$ to obtain $([\![r]\!]_d, \langle r\rangle^U)$;
2: The parties select a subset U with $|U| = 2d + 1$ and $P_1 \in U$;
3: The parties in U locally compute $\langle x \cdot y - r\rangle^U = [\![x]\!]_d \odot_U [\![y]\!]_d - \langle r\rangle^U$ and send the results to P_1;
4: P_1 reconstruct $xy - r$;
5: Let $T \in \mathcal{P}$ be a determined subset of size $n - d$ such that $P_1 \in T$. Then, P_1 send $xy - r$ to the parties in T;
6: The parties in set T compute $[\![z]\!]_d = [\![r]\!]_d + xy - r$ and store $[\![z]\!]_d$ as the output.

in T compute $z_u \leftarrow z_u - (x_j \cdot y_k)$. Eventually parties get the output z_i which forms the ASS of $x \cdot y$.

As shown in Protocol 1, based on $[\![x]\!]_d \odot_U [\![y]\!]_d$ multiplication algorithm and the function $\mathcal{F}_{\text{corRand}}$, we obtain a multiplication protocol Π_{Mult}, which makes parties only communicate twice to compute $[\![x]\!]_d \cdot [\![y]\!]_d = [\![x \cdot y]\!]_d$. The parties in U first compute the $\langle x \cdot y - r\rangle^U = [\![x]\!]_d \odot_U [\![y]\!]_d - \langle r\rangle^U$ to get the ASS of $x \cdot y - r$, then send the ASS of $x \cdot y - r$ to P_1 who can reconstruct $x \cdot y - r$. P_1 sends the $x \cdot y - r$ to the parties in a set T, $|T| = n - d$ and $P_1 \in T$. The parties in T add $x \cdot y - r$ and $[\![r]\!]_d$ to get the RSS of the $x \cdot y$.

With the multiplication protocol Π_{Mult} given above, we can implement the dot product as well. If the parties hold two vectors $[\![x_1]\!]_d, \ldots, [\![x_n]\!]_d$ and $[\![y_1]\!]_d, \ldots, [\![y_n]\!]_d$, they can compute $[\![\boldsymbol{x} \cdot \boldsymbol{y}]\!]_d = [\![\sum_{j=1}^n x_j \cdot y_j]\!]_d$. We can only perform one multiplication to obtain the inner product result. The parties first locally compute (3), and send the results to P_1 who reconstructs $\sum_{j=1}^n x_j \cdot y_j - r$. Next, as in step 5 of Protocol 1, P_1 sends the reconstructed result to everyone in a predetermined set T with $|T| = n - d$, parties in this set can locally compute $[\![\boldsymbol{x} \cdot \boldsymbol{y}]\!]_d = [\![r]\!]_d + \sum_{j=1}^n x_j \cdot y_j - r$.

$$\left\langle \sum_{j=1}^{n} x_j \cdot y_j - r \right\rangle^U = \sum_{j=1}^{n} ([\![x_j]\!] \odot_U [\![y_j]\!]) - \langle r\rangle^U. \quad (3)$$

Communication of the dot product: Let the size of the ring be ℓ bits. When P_1 is in both sets U and T, P_1 needs to receive $2d\ell$ bits and send $(n - d - 1)\ell$ bits with $|U| = 2d + 1, |T| = n - d$. So the communication of dot product requires only $(2d + n - d - 1)\ell = (n + d - 1)\ell$ bits.

We can easily extend the dot product to matrix multiplication. In the convolutional neural network, convolution calculation can be simplified to matrix multiplication [19]. For example, a convolution kernel with a shape of $(s \times k \times f \times f)$ and an image with a shape of $(i \times w \times h)$ do convolution calculation, we only need to perform $s * i$ matrix multiplications. According to the multiplication protocol mentioned above, the user of inference service inputs $\boldsymbol{x}$ and generates $[\![\boldsymbol{x}]\!]_d$. All parties now hold RSS of $\boldsymbol{x}$ and $\mathbf{M}$. In the ith linear layer, parties compute $[\![\mathbf{M_i} \cdot \boldsymbol{x_i}]\!]_d$ and output the result as the input of next nonlinear layer. $\boldsymbol{x_i}$ and $\mathbf{M_i}$ represent the input and model matrix of the ith linear layer, respectively.

3) Nonlinear Layer: We use the polynomial approximate activation function to calculate the nonlinear layer function f_{non} which is a nonlinear activation function such as ReLU and Sigmoid. Here, we mainly introduce the most widely used activation function $\text{ReLU}(x) = \max(0, x)$. We use a low-degree polynomial instead of ReLU. Although this substitution will cause a slight decrease in the modeling accuracy of the neural network, by choosing the appropriate polynomials, the effect can be reduced to almost negligible level. Next, we can use this polynomial to calculate the non-linear layer. Similarly, we compute multiplications in polynomials by using Protocol 1.

The pooling layers need to be calculated in the convolutional neural network. Average pooling is easy to be implemented by using RSS and only needs to add the values within the range of the pooling kernel and multiply by the corresponding coefficient. Max pooling needs to compare the size of the elements in the feature map within the range of the pooling kernel, according to the size of the pooling kernel, we can use the same random value r in a pooling kernel in step 3 of Protocol 1, and the elements in different pooling kernels still use different r. P_1 reconstructs the result matrix of convolution multiplication in step 4 of Protocol 1. Each element in the matrix has a structure similar to $xy - r$. P_1 can compare the size of each element locally and output the max pooling matrix. In fact, any party can perform the reconstruction operation, so we let $d + 1$ parties execute this comparison operation and broadcast the result to ensure security. If any party receives inconsistent results, then this party outputs $\perp$ to other parties.

4) Check and Output: As shown in Protocol 2, we show how the parties verify the linear computation and detect and abort the protocol when someone has cheated. Note that in our multiplication protocol, all messages are public, the only reason for the communication pattern through P_1 is to reduce overhead, but in fact the parties could send their messages in the first round directly to all parties. Our check protocol is divided into the following two steps: the first step in the check protocol is to agree on a compressed transcript of all the executions of the multiplication protocol. If all messages are consistent, the parties hold a compressed transcript of all executions, then parties can check the correctness of these messages in the second step.

Step 1. Verify message consistency: Let m be the number of multiplications. The parties first call $\mathcal{F}_{\text{coin}}$ to receive m random values $\delta_1, \ldots, \delta_m$. Then, each party broadcasts a random linear combination of messages it has sent and received separately. Specifically, each party P_i with $i \neq 1$, needs to broadcast one sent message (a linear combination of the messages sent to P_1 in step 3 of Protocol 1) if $P_i \in U$. If $P_i \in T$, P_i also broadcasts one received message (a linear combination of the messages received from P_1). At the same time, P_1 broadcasts a random linear combination of all messages received from each of the other parties and a random linear combination of the messages it sent. If there is an inconsistent message between P_1 and P_i, the fact is that either P_1 or P_i is a corrupted party. Otherwise, the parties will take the next step.

Step 2. Verify message correctness: We find that P_1's message is computed by summing all messages received from the other parties and his own calculations in step 3 of Protocol 1. Given the message sent from P_1 in step 5 of Protocol 1, the parties can compute the calculations of P_1. This implies that this step is reduced to checking local computation of $[\![x]\!]_d \odot_U [\![y]\!]_d - \langle r \rangle^U$. In step 3 of Protocol 1, each party performs a local computation over its shares of x and y, and then subtracts its additive share of r. Looking at the definition of the operation $[\![x]\!]_d \odot_U [\![y]\!]_d$ from Fig. 4, we observe that the parties compute a linear combination of their shares in this computation. Let $\gamma = \binom{n-1}{n-d-1}$ be the number of shares held by each party. We denote the shares held by P_i as $x^i_1, \ldots, x^i_\gamma, y^i_1, \ldots, y^i_\gamma$ and r^i. Then, the party P_i calculates $[\![x^i]\!]_d \odot_U [\![y^i]\!]_d - \langle r \rangle^U$ according to the (4)

$$\sum_{k=1}^{\gamma} \sum_{j=1}^{\gamma} \left(\alpha_{k,j} \cdot x^i_k \cdot y^i_j\right) - r^i \tag{4}$$

where $\alpha_{k,j}$ is a public known coefficient.

In our RSS scheme, each share is held by $n-d$ parties, we define $[\![x^i_k]\!]_d$ ($[\![y^i_j]\!]_d$) to be a consistent d-out-of- n secret sharing of x^i_k in the following way: The subset T of parties which know x^i_k sets $x^i_{k,T} = x^i_k$, whereas the other subsets define their shares to be 0. In addition, we generate the correlated randomness such that each party's additive share r^i is also secret shared across the parties. We can ask the parties to jointly compute a $2\,d$-out-of- n secret sharing of P_i 's additive share of $x \cdot y - r$, and then compute a $2\,d$-out-of-n secret sharing of the compressed message obtained in the previous step. For each $i \in U$, the parties compute (5).

$$\begin{aligned}[\![\text{msg}^i]\!]_{2\,d} &= \sum_{\ell=1}^{m} \delta_\ell \cdot [\![(x_\ell \cdot y_\ell - r_\ell)^i]\!]_{2\,d} \\ &= \sum_{\ell=1}^{m} \delta_\ell \cdot \left(\sum_{k=1}^{\gamma} \sum_{j=1}^{\gamma} \left(\alpha_{k,j} \cdot [\![x^i_{\ell,k} \cdot y^i_{\ell,j}]\!]_d\right) - [\![r^i_\ell]\!]_{2\,d} \right) \\ &\quad + [\![0]\!]_{2\,d}. \end{aligned} \tag{5}$$

$[\![0]\!]_{2\,d}$ is a secret sharing of 0 generated by $\mathcal{F}_{\text{zeroShare}}$ [11], which is added to randomize the parties' share to prevent that something can be learned from the additive shares of $(x \cdot y - r)^i$. The parties can locally compute $[\![x^i_k \cdot y^i_j]\!]_{2\,d} = [\![x^i_k]\!]_d \cdot [\![y^i_j]\!]_d$ and $[\![r^i]\!]_{2\,d} = [\![r^i]\!]_d \cdot [\![1]\!]_d$, and then locally perform addition and multiplication with the public constants. It remains to open the shared secret and check whether it equals to the additive share P_i sent. Since $3d+1 \leq n$, then $n - 2\,d \geq d+1$, which implies that in each subset of $n-2\,d$ there is at least one honest party. This means if one party sends incorrect shares, the honest party will find these incorrect messages. Thus, if all shares are consistent, the parties will hold the correct msg^i, the parties can compare it to the value obtained in the first step and check whether P_i has cheated or not.

In summary, assume that a corrupted party sent an incorrect message to an honest party in any of the multiplication protocol's executions. If it tries to publish the "correct" message in the first step of the verification, then this will result in an inconsistency with the received message published by the honest party. If it publishes the actual message that it sent, then this will result in an inconsistency with the correct message the parties compute and reconstruct in the second round of the verification. In both cases, it will cause the check protocol to send a abort message to all parties.

Communication of the Π_{Check}: Let the size of the ring be ℓ bits. In step 1 of the check protocol, each party needs to send two messages with length 2ℓ bits. In step 2, each party sends $(2d+1)\cdot\binom{n-1}{2\,d}\cdot 2\,d$ elements to other parties and broadcast one bit, but if only one party in each set sends the shares and the other parties send a single hash, the factor $2d+1$ can be removed. Therefore, the overall communication of the Π_{Check} is $\binom{n-1}{2\,d} \cdot 2d\ell + 2\ell$ bits. If Π_{Check} is executed s times, the above is multiplied by s. Note that the cost is completely independent from the number of verified multiplications. We can see that the communication of our Π_{Check} has nothing to do with the number

Protocol 2: Π_{Check} Check Protocol.

Input: The parties hold $[\![x]\!]_d$ and $[\![y]\!]_d$
Output: Accept or $\perp$

Parties broadcast their own linear combination of received and sent messages to other parties, as shown in Step 1, verify messages consistency;
if *inconsistent messages found* **then**
  output $\perp$;
for $i \in U$ **do**
  The parties locally compute $[\![\text{msg}^i]\!]_{2d}$ via Eq.5;
  For $j \in [n]$, the parties in each subset T where $|T| = n - 2 \cdot d$ and $P_j \notin T$ send their shares of msg^i to P_j;
if P_j *received contradicting shares* **then**
  sets $cons^j = 1$;
else
  sets $cons^j = 0$;
P_j broadcasts $cons^j$ to the other parties;
if *there exists a* j *such that* $cons^j = 1$ **then**
  output $\perp$;
else if *for every* j *there are* $cons^j = 0$ **then**
  The parties reconstruct msg^i for each $i \in [n]$;
  Let msg'^i be the compressed message of P_i agreed upon in the Step 1;
  if $\forall i : \text{msg}'^i = \text{msg}^i$ **then**
    output accept;
  else if $\exists i : \text{msg}'^i \neq \text{msg}^i$ **then**
    output $\perp$.

of multiplication executions, so we can only execute Π_{Check} once before outputting the inference results.

Storage analysis: In the reasoning process, each party mainly needs to store all the messages received by the computation process and the messages sent by itself. These messages are used to check the consistency and correctness of the individual computation processes in the check protocol, and when the execution of the check protocol is completed, these stored messages can be deleted. So the cumulative storage size of each party's computation process is his total communication overhead $\binom{n-1}{2d} \cdot 2d\ell + 2\ell$ bits. If the check protocol is executed once after the reasoning computation is completed, the memory that each party needs to occupy is the total communication overhead. If a party does not have enough memory to store all the messages at once, it can execute multiple check protocols to only save a portion of the messages before each check protocol.

5) Achieve Fairness: In this article, we focus on considering the possibility that a malicious party may perform an incorrect computation and the output is unavailable to all when an error is detected. We add an extra bit b to each party. Bit b is recording whether an abort message is received and initializes $b = 1$ before the protocol starts. One party sets his own $b = 0$ when he receives the abort message. When parties need to reconstruct the secret sharing, parties exchange their b, and select the majority of b to judge whether to continue to output the result. Due to the setting of the honest majority, all honest parties will agree on a value of b. If $b = 0$ means that there must be a malicious party let the protocol abort, so no one can receive the output.

C. Security Analysis

1) Security Definition: Our goal is to make only the structure of the neural network and inference result known to users. Consist with previous works [10], [15], we do not hide information about the architecture of the NN model, such as the size and type of each layer.

Definition 1: We say that Π is a secure inference protocol against a subset of malicious parties $I \subset \mathcal{P}$ with $|I| < \frac{n}{3}$ if it securely computes $\text{NN}(\cdot,\cdot)$ with the model provider inputs $\mathbf{M}$ and the user inputs $\boldsymbol{x}$.

Our protocol is resistant to attacks by malicious adversaries under two-thirds honest majority conditions, and when malicious behavior is detected, the protocol will be aborted in time to ensure that all parties can not obtain the final inference result. Our protocols are proven secure within the universal composability framework [20], these protocols maintain their security when being run in parallel and concurrently with other secure and insecure protocols.

2) Security Proof: In the offline phase of our protocol, a corresponding number of correlated randomness needs to be generated from $\mathcal{F}_{\text{corRand}}$ ideal functionality. The security proofs of $\mathcal{F}_{\text{corRand}}$, $\mathcal{F}_{\text{zeroShare}}$, and Π_{Mult} have been given in [11], where we can learn that the security of Π_{Mult} (improved DN protocol) is private in presence of malicious adversaries controlling up to d parties, due to the fact that each message sent by an honest party is masked by a new independent random additive mask. Formally, this means that the view of the adversary during the execution have the same distribution, regardless of the honest parties' inputs. In the online phase of our scheme, parties will call the $\mathcal{F}_{\text{check}}$ ideal functionality defined in Fig. 5 to check the correctness of the multiplication computation result, and if there is a check error, the parties will abort the protocol.

The $\mathcal{F}_{\text{check}}$ ideal functionality works with an ideal world adversary $\mathcal{S}$ and honest parties. $\mathcal{S}$ controls a subset $< n/3$ of corrupted parties.

1) $\mathcal{F}_{\text{check}}$ receives the inputs, randomness and sent/received messages from the honest parties, which are used to compute the inputs and randomness of the corrupted parties.
2) $\mathcal{F}_{\text{check}}$ sends the inputs and randomness from the corrupted parties and sent/received messages from the honest parties to $\mathcal{S}$.
3) Upon receiving from $\mathcal{S}$ all messages the corrupted parties claim to send/receive to/from honest parties: If the message a corrupted party P_i claims to send to an honest P_j is contradicted by the message P_j have received, then $\mathcal{F}_{\text{check}}$ sends $\perp$ to the parties. Otherwise, $\mathcal{F}_{\text{check}}$ checks whether all messages sent from corrupted parties are correct given their inputs and randomness. If it holds, then $\mathcal{F}_{\text{check}}$ sends accept to $\mathcal{S}$. Otherwise, $\mathcal{F}_{\text{check}}$ sends $\perp$ to $\mathcal{S}$. In the former case, $\mathcal{S}$ sends back either accept or reject to the $\mathcal{F}_{\text{check}}$, if $\mathcal{S}$ sends reject, $\mathcal{F}_{\text{check}}$ sends $\perp$ to the parties. In the latter case, either P_i or P_j is corrupt. Then, $\mathcal{F}_{\text{check}}$ sends $\perp$ to the parties.

Fig. 5. Ideal functionality for checking.

Lemma 1: In the ($\mathcal{F}_{\text{corRand}}$, $\mathcal{F}_{\text{zeroShare}}$)-hybrid model, the output of the $\mathcal{F}_{\text{check}}$ with an ideal world simulator $\mathcal{S}$ is computationally indistinguishable from the output of the real world, in the presence of any malicious adversary controlling up to d parties, where $d < n/3$.

Proof: Let $\mathcal{S}$ be the ideal world simulator and let $\mathcal{A}$ be the real world adversary. The only security concern in the above protocol is that something can be learned from the additive shares of $(x \cdot y - r)$, which is prevented by randomizing the sharing when adding $[\![0]\!]_{2d}$. $\mathcal{F}_{\text{check}}$ receives the sent/received messages from all parties, and the inputs and randomness of the honest parties. In the case of two-thirds honest majority, it would be sufficient to compute all the messages that corrupted parties should have sent in the check protocol. Thus, $\mathcal{F}_{\text{check}}$ can find whether any party has cheated and sent incorrect messages. In the case of cheating, there is a pair of parties P_i and P_j, at least one of them is corrupted, then the protocol outputs "$\perp$" to the parties. Note that no one is cheated in the multiplication protocol, $\mathcal{S}$ is allowed to change the output to "reject", if $\mathcal{S}$ sends "reject", $\mathcal{F}_{\text{check}}$ sends "$\perp$" to the parties. This corresponds to the case in the second step of our protocol, the corrupted parties send incorrect shares, which leads to unsuccessful verification. In the simulation, simulator $\mathcal{S}$ receives the messages sent/received in the protocol by the honest parties and also receives inputs and randomnesses of all corrupted parties. Thus, $\mathcal{S}$ can simulate step one of the check protocol. In the second step, $\mathcal{S}$ can perfectly simulate the messages sent when verifying msg^i for all i such that P_i is corrupted. This hold because all messages are a function of P_i inputs and randomnesses. These messages are known to $\mathcal{S}$, and shares distributed by $\mathcal{F}_{\text{zeroShare}}$ are played by $\mathcal{S}$. When simulating the opening of msg^i for an honest P_i, given the corrupted parties' shares which are known to $\mathcal{S}$, $\mathcal{S}$ random

selects shares for the subsets containing only honest parties. Since for the honest parties $\text{msg}'^{i} = \text{msg}^{i}$, and all shares are randomized before opening, so the honest parties' shares in the simulation are indistinguishable from their shares in the real execution. The only difference between simulation and real execution is that honest parties in the simulation accept while $\mathcal{F}_{\text{check}}$ reject, this happens when there is an incorrect message which is not checked since the random linear combination generate result is the same as the correct sent message. But we can know that over the ring $\mathbb{Z}_{2^\ell}$, the probability of this happening is only $1/|\mathbb{Z}_{2^\ell}|$. ■

VI. Implementation and Evaluation

We compare our scheme with Tetrad [5], Trident [9] and SWIFT [4] by using MNIST [21] and CIFAR-10 [22] datasets in the four-party setting. Finally, we also test our scheme in the multiparty setting.

A. Benchmarking Environment and Parameters

These schemes are tested on a Local Area Network, and instantiated with Intel Core i7-11700 CPU 2.5 GHz and 16 GB of RAM. The parties during the inference are connected by pairwise authenticated synchronous channels with 50 Mbps of bandwidth and 53 ms average round trip time. We use Python to implement our scheme and other works because Python is easier and faster to edit machine learning codes. We use a 64 bits ring ($\mathbb{Z}_{2^{64}}$) to instantiate our scheme, and use SHA-256 as hash function. To be more general, the results we record are the average of 10 tests. We use the following four networks for benchmarking.

1) *NN-1:* The first network is tested on the MNIST dataset and contains three fully connected layers consisting of 128, 128, and 10 nodes, respectively [3], [6].
2) *NN-2:* The second network is tested on the CIFAR-10 dataset and contains one convolutional layer and two fully connected layers consisting of 100 and 10 nodes [3], [5].
3) *NN-3:* The third network is LeNet [23] on the MINIST dataset, which contains two convolutional layers and three fully connected layers.
4) *NN-4:* The fourth network is VGG16 [24] on the CIFAR-10 dataset and contains 13 convolutional layers and three fully connected layers.

We evaluate our scheme by using several parameters which include online runtime, total runtime, communication, and cumulative total runtime. The online runtime and the total runtime are the maximum online runtime and maximum total runtime for $P_i \in \mathcal{P}$, respectively.

B. Evaluation

Before the performance test, we first tested the accuracy loss, in which the accuracy loss of NN-1 and NN-2 is 0.06% and 0.04%, and the accuracy loss of NN-3 and NN-4 is 0.11% and 0.23%. It can be found that the accuracy loss of the nearly larger model is larger, which is becauseNNs such as NN-4 neural network which have more layers will also have more activations during the inference process to fit, so the more the number of approximate activations, the greater the loss of accuracy.

TABLE II
Runtime (In Seconds) of Benchmarks in NN-1, NN-2, NN-3, and NN-4 in the Four-Party Setting

Network	Online runtime		Total runtime	
	Our	Tetrad	Our	Tetrad
NN-1	7.98	5.94	8.09	8.93
NN-2	8.03	5.95	8.15	8.91
NN-3	24.17	19.23	24.88	27.54
NN-4	532.22	416.56	536.78	594.17

TABLE III
Cumulative Total Runtime (In Seconds) of Benchmarks in NN-1, NN-2, NN-3, and NN-4 in the Four-Party Setting

Scheme	NN-1	NN-2	NN-3	NN-4
Our	32.36	32.60	99.52	2147.12
Tetrad	27.30	27.34	88.97	1854.07

We first compare the communication between Tetrad, Trident, SWIFT, and our scheme. In Fig. 6, we show the communication of these schemes in four types of neural networks in the four-party setting. It can be seen in NN-1 and NN-2 with smaller models, the communication of our scheme is very close to that of Tetrad. However, in NN-3 and NN-4 with larger models, our scheme saves 79% and 33% communication than Tetrad, respectively, and 80% and 40% than Trident/SWIFT. Fig. 6 shows the test results on the MNIST and CIFAR-10 datasets, which proves that our scheme is effective for both two datasets and our scheme is more suitable for NNs with more parameters and deeper network structure. NN-4 has more number of computations and each party needs to save the messages sent and received in each computation in the memory for the check protocol, which may make the memory overflow. So multiple check protocols are executed in the NN-4, which make the communication of NN-4 is not saved as much as NN-3.

Since the time overhead is not the strong point of our scheme, we only compare it with the most advanced scheme Tetrad, as shown in Table II. Our scheme takes less total runtime in every network, and the total runtime of our scheme improved by $8\% - 10\%$. Tetrad has a longer inference time as the use of conversion between arithmetic and garbled world, and this conversion also results in nonlinear layers having more communication. Our online runtime (running time after users input private data) is about 20% slower than Tetrad, which is because our offline phase does not preprocess each layer of the neural network. As shown in Table III, our scheme has longer cumulative runtime (sum of the up-time of all parties), which is because all parties need to run protocol synchronously in our scheme, and they finish running almost at the same time. In contrast, there are only three main operators in Tetrad. When we instantiate our scheme, we call the matrix multiplication as the smallest function, we only use the multithreading in convolutional layers, and each thread is used to process a convolution calculation. So the runtime of NN-2 is similar to that of NN-1. In Fig. 7, we present the performances of runtime and communication of our scheme on the NN-1 network when $n \in \{4, 5, 6, 7, 8, 9\}$. From the graph, we can see that both

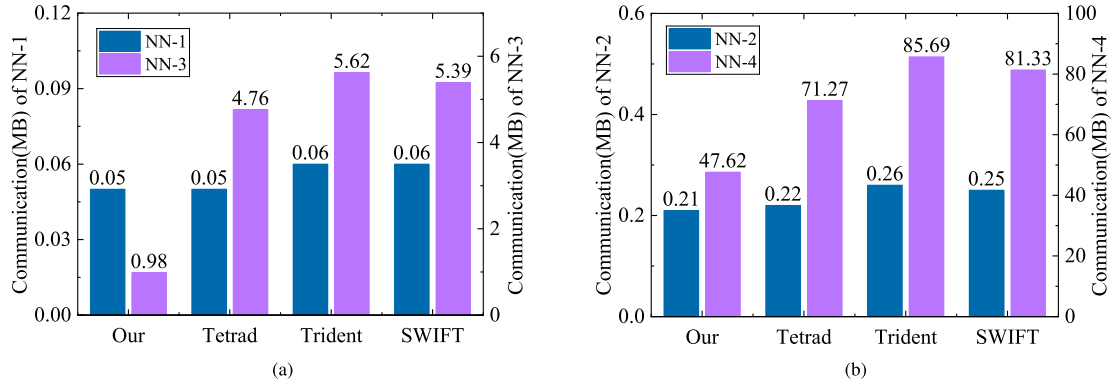


Fig. 6. Communication (in MB) of benchmarks in NN-1, NN-2, NN-3, and NN-4 in the four-party setting. (a) MNIST. (b) CIFAR-10.

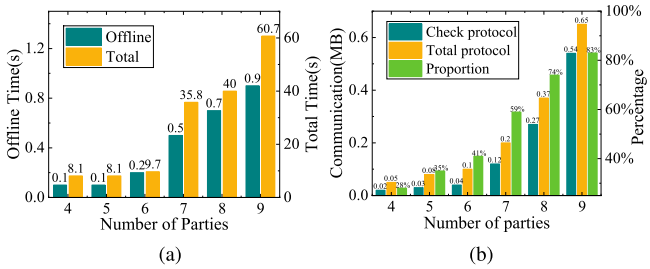


Fig. 7. Runtime and communication in different numbers of parties on NN-1. (a) Runtime. (b) Communication.

the inference time and communication increase exponentially as n grows. When there are $k = \binom{n}{n-d}$ secret shares, each party has $t = \binom{n-1}{d}$ shares, and $t = k - \binom{d-1}{n-1}$. The multiplication of each SS requires the party to calculate $(t+1)t/2$ arithmetic multiplications locally to implement our Π_{Mult} protocol. When the k is larger, the computation required for each secret shared multiplication is also larger, i.e., the time overhead is also larger. So the complexity of the time overhead is $O(k^2)$. The factors of affecting communication overhead are the size of $|U| = 2d + 1$ and n. Our multiplication protocol requires each party in U to communicate two messages, so the larger $|U|$ the larger the communication overhead. The green areas in Fig. 7(b) represent the communication proportion of the check protocol in the total scheme, which has increased from 28% to 83%. No matter how much n is, we only run the check protocol once. However, the messages are verified by check protocol increase exponentially as n increases, which generates more communication than the multiplication protocol.

Based on the above test results, we believe that our scheme has advantages in the following scenarios: Multiparty executes the inference scheme jointly in multiuser multimodel provider setting (our scheme adapts to more parties); the network speed is relatively slow (our scheme has less communication); the model is relatively large (our scheme saves communication more significantly in large models).

VII. Conclusion

We have proposed a multiparty secure inference scheme with an adaptive number of parties. Our scheme is able to perform secure inference with a two-thirds honest majority setting when there are n parties involved. We conducted evaluations of the security and performance of our scheme, our empirical results demonstrate that our scheme still has practicality even when extend to more parties. Moving forward, we intend to design a more efficient online phase by shifting some calculations from the online phase to the offline phase, which would improve the overall experience of using secure inference services. In addition, we plan to integrate security training and secure inference. These represent promising future research directions that could lead to even more powerful and versatile secure inference schemes.

References

[1] L. Da Xu, W. He, and S. Li, "Internet of Things in industries: A survey," *IEEE Trans. Ind. Informat.*, vol. 10, no. 4, pp. 2233–2243, Nov. 2014.
[2] D. Evans et al., "A pragmatic introduction to secure multi-party computation," *Foundations Trends Privacy Secur.*, vol. 2, no. 2/3, pp. 70–246, 2018.
[3] P. Mohassel and P. Rindal, "ABY^3: A mixed protocol framework for machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 35–52.
[4] N. Koti, M. Pancholi, A. Patra, and A. Suresh, "Swift: Super-fast and robust privacy-preserving machine learning," in *Proc. USENIX Secur. Symp.*, 2021, pp. 2651–2668.
[5] N. Koti, A. Patra, R. Rachuri, and A. Suresh, "Tetrad: Actively secure 4PC for secure training and inference," *Netw. Distrib. Syst. Secur.*, pp. 24–28, 2022.
[6] A. Patra and A. Suresh, "Blaze: Blazing fast privacy-preserving machine learning," arXiv preprint 2020, *arXiv:2005.09042*.
[7] A. K. Sangaiah, D. V. Medhane, T. Han, M. S. Hossain, and G. Muhammad, "Enforcing position-based confidentiality with machine learning paradigm through mobile edge computing in real-time industrial informatics," *IEEE Trans. Ind. Informat.*, vol. 15, no. 7, pp. 4189–4196, Jul. 2019.
[8] P. Mohassel and Y. Zhang, "SecureML: A system for scalable privacy-preserving machine learning," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 19–38.
[9] S. Rachuri, A. Suresh, and H. Chaudhari, "Trident: Efficient 4PC framework for privacy preserving machine learning," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–18.
[10] R. Lehmkuhl, P. Mishra, A. Srinivasan, and R. Popa, "Muse: Secure inference resilient to malicious clients," in *Proc. USENIX Secur. Symp.*, 2021, pp. 2201–2218.
[11] A. Dalskov, D. Escudero, and A. Nof, "Fast fully secure multi-party computation over any ring with two-thirds honest majority," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 653–666.
[12] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "Gazelle: A low latency framework for secure neural network inference," in *Proc. USENIX Secur. Symp.*, 2018, pp. 1651–1669.

[13] J. Liu, M. Juuti, Y. Lu, and N. Asokan, "Oblivious neural network predictions via minionn transformations," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 619–631.
[14] I. Damgård, V. Pastro, N. Smart, and S. Zakarias, "Multiparty computation from somewhat homomorphic encryption," in *Proc. Annu. Cryptol. Conf.*, 2012, pp. 643–662.
[15] P. Mishra, R. Lehmkuhl, A. Srinivasan, W. Zheng, and R. Popa, "Delphi: A cryptographic inference service for neural networks," in *Proc. USENIX Secur. Symp.*, 2020, pp. 2505–2522.
[16] N. Chandran, D. Gupta, S. Obbattu, and A. Shah, "SIMC: Ml inference secure against malicious clients at semi-honest cost," in *Proc. USENIX Secur. Symp.*, 2022, pp. 1361–1378.
[17] R. Cramer, I. Damgård, and Y. Ishai, "Share conversion, pseudorandom secret-sharing and applications to secure computation," in *Proc. Theory Cryptogr.*, 2005, pp. 342–362.
[18] G. Oded, "Foundations of Cryptography," in *Basic Applications*, vol. 2, USA: Cambridge University Press, 2009.
[19] A. Karpathy, "Convolutional Neural Networks for Visual Recognition, 2019. [Online]. Available: http://cs231n.github.io/optimization-1/
[20] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *Proc. IEEE Symp. Foundations Comput. Sci.*, 2001, pp. 136–145.
[21] Y. LeCun and C. Cortes, "MNIST Handwritten Digit Database," 2010. [Online]. Available: http://yann.lecun.com/exdb/mnist/
[22] A. Krizhevsky, V. Nair, and G. Hinton, "The CIFAR-10 dataset," 2014. [Online]. Available: http://www.cs.toronto.edu/kriz/cifar.html
[23] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
[24] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.

Jie Lin received the B.E. degree in civil engineering from the College of Civil Engineering, Taiyuan University of Technology, Taiyuan, China, in 2018. He is currently working toward the M.E degree in cyber security with the School of Cyber Engineering, Xidian University, Shaanxi, China.

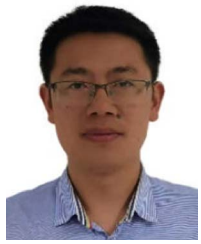
His research interests include machine learning, information security, and applied cryptography.

Yinbin Miao (Member IEEE) received the B.E. degree in communication engineering from the Department of Telecommunication Engineering, Jilin University, Changchun, China, in 2011, and the Ph.D. degree in information security from the Department of Telecommunication Engineering, Xidian University, Shaanxi, China, in 2016.

From 2018 to 2019, he was a Postdoctor with Nanyang Technological University, Nanjing, China, and from 2019 to 2021, a Postdoctor with the City University of Hong Kong, Hong Kong. He is currently a Professor with the Department of Cyber Engineering, Xidian University. His research interests include information security and applied cryptography.

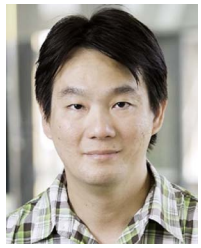
Linfeng Wei received the B.S. degree in computer science and technology from the School of Information Science and Technology and the Ph.D. degree from the School of Cyber Security, Jinan University, Guangzhou, China, in 2010 and 2020, respectively

He is currently an Associate Professor with the School of Cyber Security, and Assistant Director of the National and Local Joint Engineering Research Center for Cyber Security Detection and Protection, Jinan University. His current research interests include cyber threat intelligence, data security and privacy preserving, AI security, etc.



Tao Leng received the master's degree in computer application technology from Sichuan Normal University, Chengdu, China, in 2012. He is currently working toward the Ph.D. degree in cyberspace security with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China.

He serves as an Associate Professor and Master's Supervisor with Sichuan Police College, Luzhou, China. His research field is cyberspace security. His research interests include threat hunting, advanced threat detection, forensic analysis, and graph neural networks.



Kim-Kwang Raymond Choo (Senior Member, IEEE) received the Ph.D. degree in information security from the /Queensland University of Technology, Brisbane, Australia, in 2006.

He is the founding co-Editor-in-Chief for *ACM Distributed Ledger Technologies: Research & Practice*.

Dr. Choo currently holds the Cloud Technology Endowed Professorship at The University of Texas at San Antonio, San Antonio, TX, USA. He is the founding Chair of IEEE Technology and Engineering Management Society Technical Committee (TC) on Blockchain and Distributed Ledger Technologies.